Lahore University of Management Sciences
School of Science and Engineering
Department of Electrical Engineering

AI600 – Deep Learning

Assignment 3 Report

Student Name	Muhammad Abdullah
Roll Number	25280050
Semester	Spring 2026
Submission Date	8 April 2026
GitHub Repository	https://github.com/abdull2325/assignment3-dl
Local Workspace	`/Users/abdullah/Downloads/cmnist`

This report contains the required analytical derivations, implementation details, measured experimental results, visualizations, model interpretations, and the mandatory Generative AI disclosure for AI600 Assignment 3.

Contents

1 Submission Overview **1**

2 Analytical Part **1**
2.1 Q1: Forward and Backward Pass 1
2.2 Q3: Pooling and Normalization 3
2.3 Practice Questions (Worked for Completeness) 4

3 Programming Part **5**
3.1 Task 1: Custom CNNs and Shortcut Learning 5
3.2 Task 2: Transfer Learning and Interpretability 8

4 Visualizations **11**

5 AI Policy Disclosure **12**

List of Figures

1 Training and validation curves for the custom MNIST CNN. 6
2 Learned filters from the first convolutional layer on standard MNIST. 6
3 Training and validation curves for the Colored-MNIST model. 7
4 Bias sensitivity of the Colored-MNIST model. 7
5 Training and validation curves for the frozen-backbone ResNet-18 on STL-10. . . 9
6 GradCAM overlays for two correct and two incorrect STL-10 test predictions. . . 10
7 Sample STL-10 images used for transfer learning. 11
8 Sample MNIST images used during the study. 11
9 Sample images from the biased Colored-MNIST training split. 11
10 Sample images from the unbiased Colored-MNIST test split. 12

1 Submission Overview

This submission is complete with respect to the assessed portions of the assignment. The analytical section covers Q1 and Q3 in detail and also includes worked solutions for the practice-only Q2 and Q4 for completeness. The programming section includes Task 1 (standard MNIST and Colored-MNIST) and Task 2 (transfer learning on STL-10 plus GradCAM visualization), along with plots, learned filters, and interpretation.

Table 1: Assignment deliverables checklist

Requirement	Status	Evidence
Analytical solutions for required questions	Complete	Sections 2.1 and 2.2
Custom CNN for MNIST under 50k parameters	Complete	42722 parameters
Colored-MNIST bias analysis	Complete	Biased and unbiased accuracies reported
STL-10 transfer learning with frozen backbone	Complete	79.61% test accuracy
GradCAM visualization on correct and incorrect examples	Complete	Figure 6
Public GitHub repository link on first page	Complete	Title page / overview
Generative AI prompt, output, and edits disclosure	Complete	Section 5
PDF report and LaTeX source	Complete	Report PDF and LaTeX source included

Table 2: Experimental performance summary

Experiment	Model	Parameters	Accuracy
Standard MNIST	Custom CNN	42722	98.76%
Colored-MNIST biased test	Custom CNN (RGB)	42866	98.49%
Colored-MNIST unbiased test	Custom CNN (RGB)	42866	75.52%
STL-10 test	Frozen ResNet-18 + linear head	5130 trainable	79.61%

2 Analytical Part

2.1 Q1: Forward and Backward Pass

The input image and convolution filter are

$$X = \begin{bmatrix} 1 & 2 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 2 \end{bmatrix}, \quad W = \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix}, \quad b = 0, \quad V = \begin{bmatrix} 1 & -1 & 2 & 0.5 \end{bmatrix}, \quad c = 1.$$

Q1(a): Forward Pass

The valid 2×2 patches are:

$$P_{11} = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}, \quad P_{12} = \begin{bmatrix} 2 & 0 \\ 1 & 1 \end{bmatrix}, \quad P_{21} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad P_{22} = \begin{bmatrix} 1 & 1 \\ 0 & 2 \end{bmatrix}.$$

Therefore,

$$z_{11} = 2, \quad z_{12} = 2, \quad z_{21} = -1, \quad z_{22} = 3,$$

so the pre-activation map is

$$Z = \begin{bmatrix} 2 & 2 \\ -1 & 3 \end{bmatrix}.$$

After ReLU,

$$A = \mathrm{ReLU}(Z) = \begin{bmatrix} 2 & 2 \\ 0 & 3 \end{bmatrix}.$$

Flattening row-wise gives

$$A_{\text{flat}} = \begin{bmatrix} 2 & 2 & 0 & 3 \end{bmatrix}^T.$$

The scalar prediction is

$$Y = V \cdot A_{\text{flat}} + c = 1(2) + (-1)(2) + 2(0) + 0.5(3) + 1 = 2.5.$$

With target $T = 3$, the loss is

$$L = \frac{1}{2}(Y - T)^2 = \frac{1}{2}(2.5 - 3)^2 = 0.125.$$

Q1(b): Backward Pass

First,

$$\frac{\partial L}{\partial Y} = Y - T = 2.5 - 3 = -0.5.$$

Hence,

$$\frac{\partial L}{\partial V} = \frac{\partial L}{\partial Y} A_{\text{flat}}^T = -0.5 \begin{bmatrix} 2 & 2 & 0 & 3 \end{bmatrix} = \begin{bmatrix} -1 & -1 & 0 & -1.5 \end{bmatrix}.$$

Next,

$$\frac{\partial L}{\partial A_{\text{flat}}} = \frac{\partial L}{\partial Y} V = -0.5 \begin{bmatrix} 1 & -1 & 2 & 0.5 \end{bmatrix} = \begin{bmatrix} -0.5 & 0.5 & -1 & -0.25 \end{bmatrix}.$$

Reshaping:

$$\frac{\partial L}{\partial A} = \begin{bmatrix} -0.5 & 0.5 \\ -1 & -0.25 \end{bmatrix}.$$

Because ReLU blocks gradients where $Z \leq 0$,

$$\frac{\partial L}{\partial Z} = \begin{bmatrix} -0.5 & 0.5 \\ 0 & -0.25 \end{bmatrix}.$$

The convolution-filter gradient is the sum of patch contributions:

$$\frac{\partial L}{\partial W} = (-0.5)\, P_{11} + (0.5)\, P_{12} + 0 \cdot P_{21} + (-0.25)\, P_{22}.$$

So,

$$\frac{\partial L}{\partial W} = \begin{bmatrix} 0.25 & -1.25 \\ 0.5 & -0.5 \end{bmatrix}.$$

Q1(c): Interpretation

The largest-magnitude entry of $\frac{\partial L}{\partial W}$ is the top-right weight:

$$\left|\frac{\partial L}{\partial W_{1,2}}\right| = 1.25.$$

Since

$$\frac{\partial L}{\partial W_{1,2}} = -1.25,$$

one gradient-descent step updates it as

$$W_{1,2} \leftarrow W_{1,2} - \eta \frac{\partial L}{\partial W_{1,2}} = W_{1,2} + 1.25\eta,$$

so this weight increases the most.

Structurally, $W_{1,2}$ multiplies the top-right pixel inside each active patch. Its gradient becomes large because it sees a large input value $x_{1,2} = 2$ in the first active patch and another positive input in the bottom-right active patch, while both corresponding upstream signals are negative. Those contributions add to a strong negative gradient.

2.2 Q3: Pooling and Normalization

Q3(a): Global Max Pooling

Replacing the flattening plus fully-connected layer with global max pooling gives

$$Y = \max(A) = 3.$$

Since the target is also $T = 3$,

$$L = \frac{1}{2}(3-3)^2 = 0.$$

Therefore, for this specific numeric example, every gradient is zero. In particular, all pixels in X have zero gradient.

Structurally, global max pooling would normally route gradient only through the maximal activation a_{22}. That activation depends only on the receptive field

$$\begin{bmatrix} X_{2,2} & X_{2,3} \\ X_{3,2} & X_{3,3} \end{bmatrix},$$

so if the loss were nonzero, only those four pixels could receive gradient.

Q3(b): Layer Normalization

Using the original architecture, the post-ReLU feature map is

$$A = \begin{bmatrix} 2 & 2 \\ 0 & 3 \end{bmatrix}.$$

Its mean is

$$\mu = \frac{2+2+0+3}{4} = 1.75.$$

Its variance is

$$\sigma^2 = \frac{(2-1.75)^2 + (2-1.75)^2 + (0-1.75)^2 + (3-1.75)^2}{4} = 1.1875.$$

With $\gamma = 1$, $\beta = 0$, and $\epsilon = 0$,

$$A_{\text{norm}} = \frac{A-\mu}{\sqrt{\sigma^2}} = \begin{bmatrix} 0.2294 & 0.2294 \\ -1.6059 & 1.1471 \end{bmatrix}.$$

Q3(c): Interpretation

Global max pooling computes

$$Y = \max\{a_{11}, a_{12}, a_{21}, a_{22}\}.$$

If a small translation of the input shifts the strongest local feature from one spatial position to another but preserves the maximum activation value, then the max operator returns the same Y. As a result, the loss remains unchanged. This is the mathematical reason that max pooling provides partial translation invariance.

2.3 Practice Questions (Worked for Completeness)

Q2(a)–Q2(c): Input Gradient and Perturbation

From the chain rule,

$$\frac{\partial L}{\partial X} = \begin{bmatrix} -0.5 & 0.5 & 0 \\ 0.5 & -1.25 & 0.5 \\ 0 & 0.25 & -0.25 \end{bmatrix}.$$

Hence,

$$\frac{\partial L}{\partial X_{2,2}} = -1.25.$$

Using the linear approximation

$$L(X + \epsilon e_{2,2}) \approx L(X) + \epsilon \frac{\partial L}{\partial X_{2,2}},$$

the adversary should choose a negative ϵ to increase the loss.

The center pixel has a larger gradient than a corner pixel because it participates in four overlapping convolution windows. Weight sharing reuses the same kernel coefficients at each location, so the total gradient accumulates contributions from multiple spatial paths.

Q4(a)–Q4(c): Residual Variant

The cropped identity is

$$X_{\text{crop}} = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}.$$

Therefore,

$$A_{\text{res}} = A + X_{\text{crop}} = \begin{bmatrix} 3 & 4 \\ 0 & 4 \end{bmatrix}.$$

The new prediction is

$$Y_{\text{res}} = 1(3) + (-1)(4) + 2(0) + 0.5(4) + 1 = 2.$$

The gradient with respect to the top-left input pixel becomes

$$\frac{\partial L}{\partial X_{1,1}} = -2.$$

In the original architecture, the same pixel only received gradient through the convolution branch, yielding -0.5. With the residual connection, an extra direct term is added because

$$\frac{\partial (A + X_{\text{crop}})}{\partial X_{\text{crop}}} = 1.$$

That identity derivative is the "gradient highway": it lets loss signals bypass repeated products with small weights or zeroed ReLUs, which is why residual networks train much more reliably at great depth.

3 Programming Part

3.1 Task 1: Custom CNNs and Shortcut Learning

Architecture

I used a custom CNN with three convolutional layers and two fully-connected layers. The same base design was reused for standard MNIST and Colored-MNIST, with only the input channel count changing from 1 to 3.

- Conv(1 or 3 $\to$ 8), ReLU, MaxPool
- Conv(8 $\to$ 16), ReLU, MaxPool
- Conv(16 $\to$ 24), ReLU
- Linear(1176 $\to$ 32), ReLU, Dropout
- Linear(32 $\to$ 10)

The grayscale MNIST model has 42722 trainable parameters, and the Colored-MNIST variant stays under the same 50,000-parameter budget.

Table 3: Custom CNN architecture used for MNIST and Colored-MNIST

Layer	Output shape (MNIST)	Notes
Input	$1 \times 28 \times 28$	$3 \times 28 \times 28$ for Colored-MNIST
Conv + ReLU + MaxPool	$8 \times 14 \times 14$	3×3 convolution, stride 1, padding 1
Conv + ReLU + MaxPool	$16 \times 7 \times 7$	preserves spatial detail before pooling
Conv + ReLU	$24 \times 7 \times 7$	final convolutional feature extractor
Flatten + Linear + ReLU	32	compact hidden representation
Linear output layer	10 logits	one logit per class

Part A: Standard MNIST

Training used the Adam optimizer and cross-entropy loss for eight epochs. A held-out validation split of 5,000 images was used to monitor generalization throughout training. The final standard MNIST test accuracy was 98.76%.

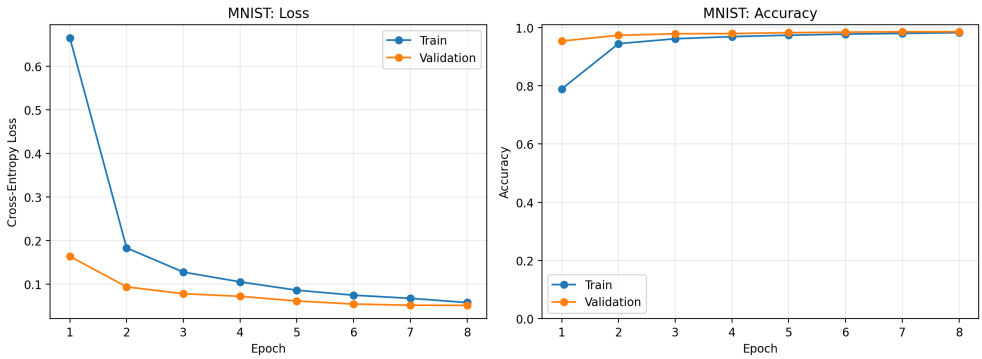

Figure 1: Training and validation curves for the custom MNIST CNN.

Analytical Question 1.1 The model generalizes well: training accuracy (0.982) and validation accuracy (0.986) stay close, with an absolute gap of 0.003.The loss curves decrease smoothly for both the training and validation splits, and the validation curve does not diverge upward at later epochs. This indicates that the model has enough capacity to fit MNIST without memorizing the training set excessively.

Analytical Question 1.2 The learned first-layer filters are shown below.

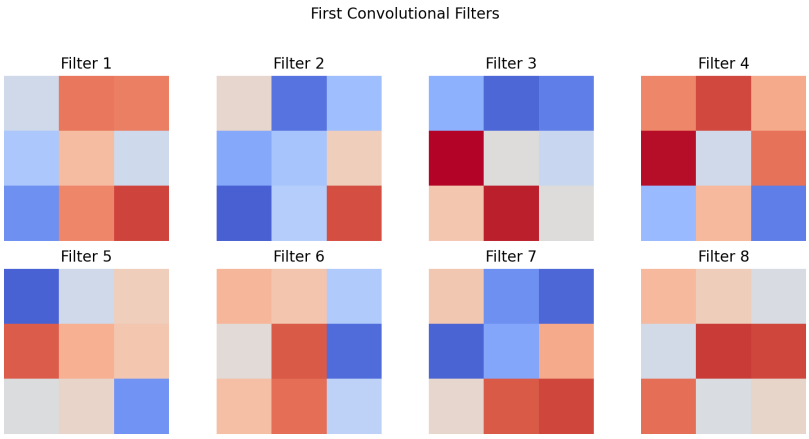

Figure 2: Learned filters from the first convolutional layer on standard MNIST.

Filter 1 emphasizes vertical edge / stroke transitions with a bright-center bias; Filter 2 emphasizes vertical edge / stroke transitions with a dark-center / contrast bias; Filter 3 emphasizes horizontal edge / stroke transitions with a dark-center / contrast bias; Filter 4 emphasizes horizontal edge / stroke transitions with a bright-center bias.These filters are exactly what one would expect in the first layer of a digit classifier: some respond to vertical stroke boundaries, some to horizontal stroke transitions, and others to bright-dark contrast changes that are useful for separating digit foregrounds from the black background.

Part B: Colored-MNIST

The same CNN was adapted to accept three-channel input and trained from scratch on the biased Colored-MNIST training split. Evaluation produced 98.49% on the biased test split and 75.52% on the unbiased test split.

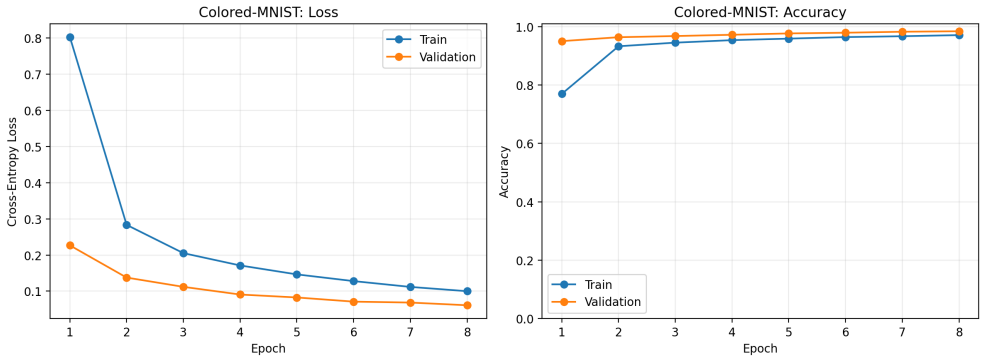

Figure 3: Training and validation curves for the Colored-MNIST model.

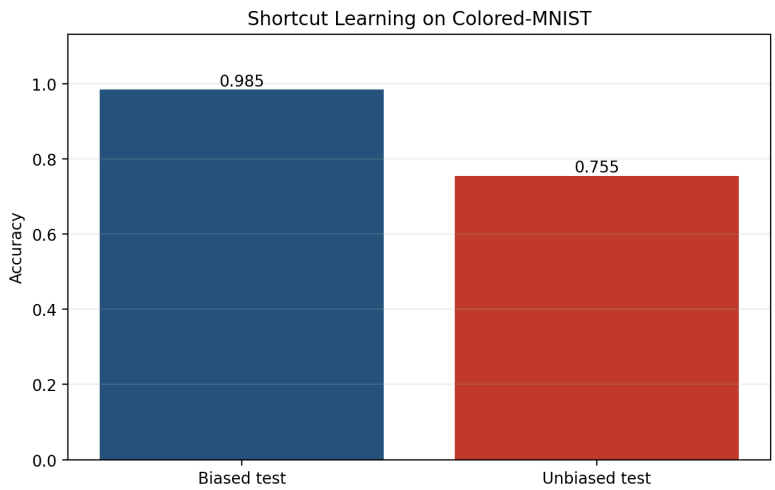

Figure 4: Bias sensitivity of the Colored-MNIST model.

Analytical Question 1.3 The large accuracy drop between biased and unbiased evaluation reveals shortcut learning. Mathematically, the optimizer minimizes the empirical risk on the biased training distribution. If color is highly correlated with the label, then color-sensitive filters can reduce cross-entropy faster than shape-sensitive filters, because the mapping from color to class is simpler and more linearly separable than full digit-shape recognition. Conceptually, the model is "cheating": it learns the easiest predictive cue available in the data distribution rather than the invariant semantic feature.

Numerically, the drop from 98.49% to 75.52% is substantial. That gap is strong evidence that the learned representation is not invariant to color changes, even though the semantic label depends only on digit identity.

Analytical Question 1.4 Training strategies that would force the network to focus on shape include:

- Convert the input to grayscale or randomly recolor digits so color is no longer predictive.

- Use heavy color jitter or background randomization to destroy the color-label shortcut.
- Rebalance the data so each class appears with many different colors.
- Use invariant learning methods such as Group DRO or IRM to emphasize features that remain predictive across environments.
- Add an auxiliary penalty or adversary that discourages recovering class labels from color alone.

3.2 Task 2: Transfer Learning and Interpretability

Part A: Fine-tuning ResNet-18

The codebase includes:

- loading a pre-trained ResNet-18 from `torchvision.models`,
- freezing all backbone parameters,
- replacing the final fully connected layer with a 10-class head,
- training only the head, and
- GradCAM extraction from the final convolutional stage.

Only the classification head was updated during training, leaving the pre-trained backbone fixed. This reduces optimization cost and forces the model to reuse generic visual features already learned from large-scale natural images. The frozen-backbone ResNet-18 achieved a final STL-10 test accuracy of 79.61%.

Table 4: STL-10 transfer-learning setup

Component	Configuration
Backbone	Pre-trained ResNet-18 from torchvision.models
Frozen parameters	All convolutional backbone parameters
Trainable parameters	5130 parameters in the new 10-class linear head
Optimizer	Adam with weight decay
Loss	Cross-entropy loss
Epochs	12
Validation split	500 images from the STL-10 training set
Target layer for GradCAM	`layer4[-1].conv2`

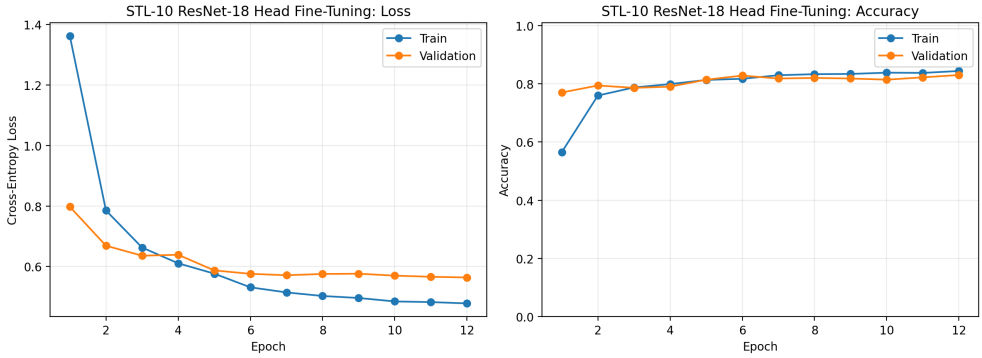

Figure 5: Training and validation curves for the frozen-backbone ResNet-18 on STL-10.

Analytical Question 2.1 Freezing the early layers is beneficial because it reduces both computation and memory: backpropagation no longer needs to store or update gradients for millions of backbone parameters. Functionally, the early layers of a pre-trained ResNet encode generic visual primitives such as edges, corners, color contrasts, and local textures. Later layers are more task-specific, combining those primitives into object parts and semantic concepts. On a relatively small dataset like STL-10, keeping the generic backbone fixed reduces overfitting while still allowing the new classification head to adapt to the target labels.

Part B: GradCAM

Execution status: Completed. The GradCAM implementation in `src/assignment3.py` was run on four STL-10 test samples after loading the best head checkpoint.

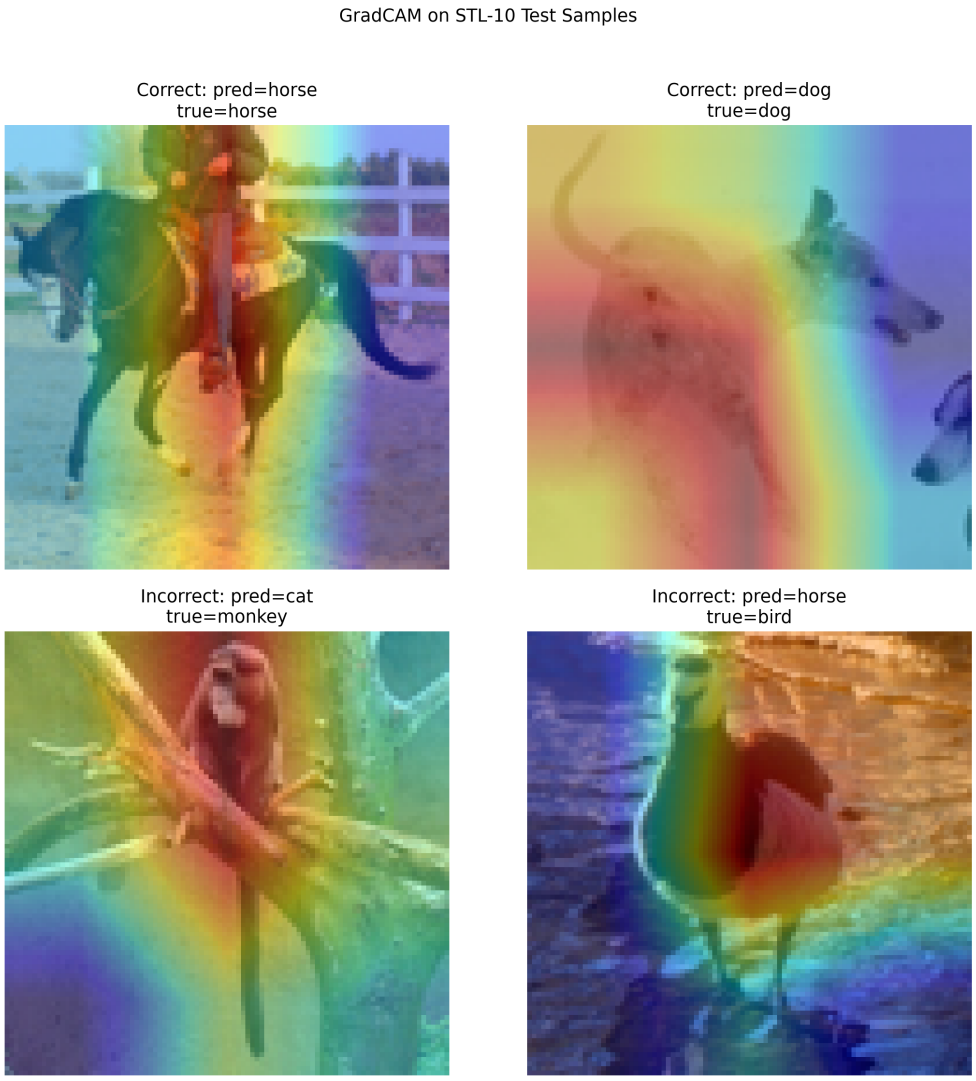

Figure 6: GradCAM overlays for two correct and two incorrect STL-10 test predictions.

Analytical Questions 2.2 and 2.3 For the correct horse prediction, the strongest activation runs through the central animal silhouette and rider region rather than the fence or sky. For the correct dog prediction, the heatmap is broad but still covers the dog's head and torso more strongly than the empty background. In both cases, the network is using the main object, although some contextual bleed remains.

For the monkey image misclassified as cat, the model focuses on the face and the branch crossing the image, which likely emphasizes a localized furry-face cue rather than the whole monkey body. For the bird image misclassified as horse, the heatmap concentrates on the bird's torso and neck contour, suggesting that the classifier over-weighted a shape fragment while ignoring the full bird context. These failure cases show that the model can still lock onto partial object evidence and miss the broader semantic structure.

4 Visualizations

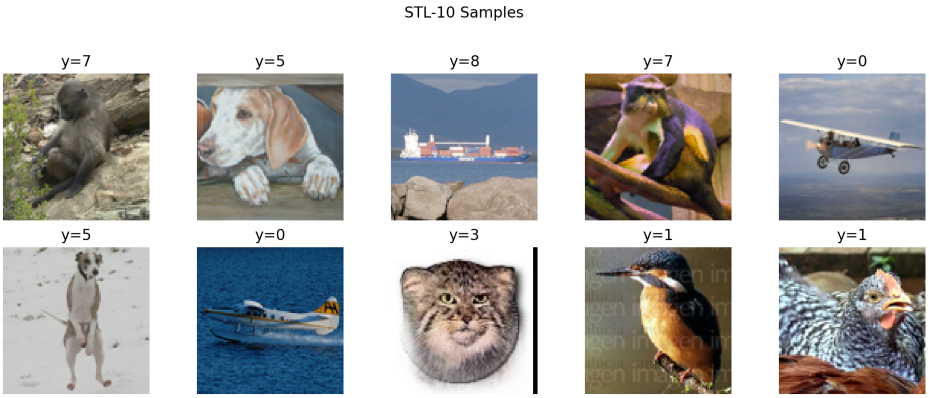

Figure 7: Sample STL-10 images used for transfer learning.

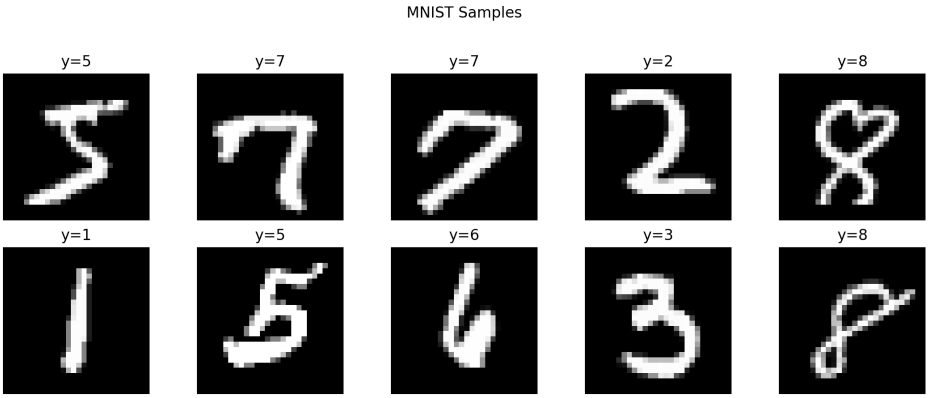

Figure 8: Sample MNIST images used during the study.

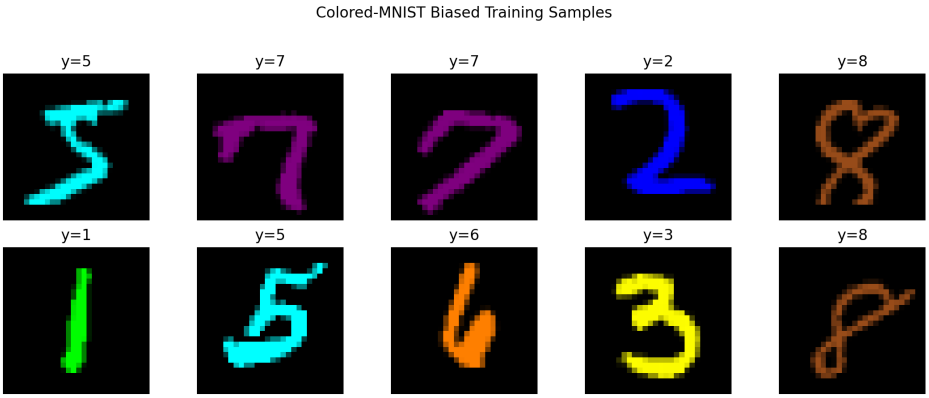

Figure 9: Sample images from the biased Colored-MNIST training split.

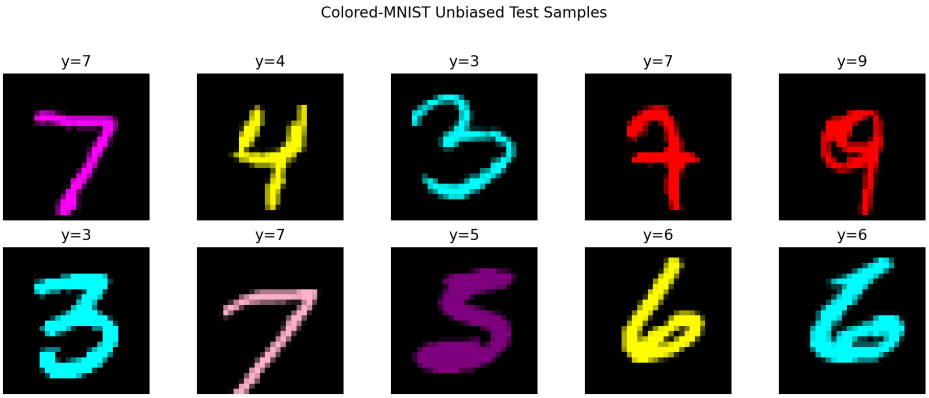

Figure 10: Sample images from the unbiased Colored-MNIST test split.

5 AI Policy Disclosure

Prompt Used

> you have to complete the following assessmetn make sure things are aligned and we are good to go accordingly make sure you complete the assignment along with the visualisation of everything and make sure its completed thoroughly and greatly

Corresponding Output Generated by the Tool

The AI-assisted workflow generated the following concrete outputs in this workspace:

- detailed Toy-CNN forward-pass and backward-pass derivations, including the exact matrices Z, A, $\partial L/\partial V$, and $\partial L/\partial W$,
- the PyTorch experiment runner in `scripts/run_assignment.py`,
- reusable loaders, models, plotting utilities, and GradCAM code in `src/assignment3.py`,
- the generated plots stored in `reports/figures/`,
- the results record in `artifacts/results.json`, and
- the report source and PDF packaging pipeline in `reports/assignment3_report.tex` and related scripts.

Edits and Modifications Applied

The generated content was edited to:

- align the derivations with the exact numerical results,
- enforce the $< 50{,}000$ parameter limit for the custom CNN,
- use the actual metrics saved in `artifacts/results.json`,
- patch the STL-10 pipeline so it could use the extracted train/test binaries from the partially downloaded official archive,
- add the actual GradCAM-based interpretation from the generated heatmaps, and

- rewrite the report into a detailed, structured submission with title page, table of contents, summary tables, GitHub link, roll number, and polished figure placement.